Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РУБЕЖНЫЙ КОНТРОЛЬ №1

«Обработка данных со связями один-ко-многим и многие-ко-многим на языке Python»

ДИСЦИПЛИНА: «Парадигмы и Конструкции Языков Программирования»

Выполнил: студент гр. ИУ5-33Б _____ (Федяев А. С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Цель: закрепление навыков объектно-ориентированного и функционального программирования на языке Python путём разработки классов данных со связями «один-ко-многим» и «многие-ко-многим» и выполнения запросов к ним.

Задачи:

1. Создать два класса данных в соответствии с вариантом 16 предметной области («Книга» и «Книжный магазин») и связующий класс «Книги книжного магазина», реализующие связи «один-ко-многим» и «многие-ко-многим».
2. Сформировать списки объектов классов, содержащих тестовые данные, связанные по идентификаторам (первичным и вторичным ключам), и ввести количественный признак – стоимость книги.
3. Разработать запрос B1 (связь «один-ко-многим»): вывести список всех книг, у которых название начинается с буквы «А», и названия их книжных магазинов.
4. Разработать запрос B2 (связь «один-ко-многим»): вывести список книжных магазинов с минимальной стоимостью книг в каждом магазине, отсортированный по минимальной стоимости.
5. Разработать запрос B3 (связь «многие-ко-многим»): вывести список всех связанных книг и книжных магазинов, отсортированный по книгам (сортировка по магазинам произвольная).

Выполнение работы

1. Описание классов данных «Книга», «Книжный магазин» и «Книги книжного магазина», а также формирование списков тестовых данных со связями по идентификаторам представлены на рисунке 1.

```python
class Book:
    def __init__(self, id, title, price, shop_id):
        self.id = id
        self.title = title
        self.price = price
        self.shop_id = shop_id


class Shop:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class ShopBook:
    def __init__(self, shop_id, book_id):
        self.shop_id = shop_id
        self.book_id = book_id

shops = [
    Shop(1, 'Читай-город'),
    Shop(2, 'Библио-Глобус'),
    Shop(3, 'Московский Дом Книги'),
    Shop(11, 'Республика'),
    Shop(22, 'Лабиринт'),
    Shop(33, 'Буквоед'),
]

books = [
    Book(1, 'Алгоритмы: построение и анализ', 2400, 1),
    Book(2, 'Архитектура компьютера', 1950, 2),
    Book(3, 'Изучаем Python', 1800, 2),
    Book(4, 'Анализ данных в Python', 1450, 3),
    Book(5, 'Чистый код', 1200, 3),
    Book(6, 'Совершенный код', 2100, 3),
]

shops_books = [
    ShopBook(1, 1),
    ShopBook(2, 2),
    ShopBook(2, 3),
    ShopBook(3, 4),
    ShopBook(3, 5),
    ShopBook(3, 6),
    ShopBook(11, 1),
    ShopBook(11, 4),
    ShopBook(22, 2),
    ShopBook(22, 5),
    ShopBook(33, 3),
    ShopBook(33, 6),
]
```

Рисунок 1 – Код классов данных и списков тестовых объектов (RK1.py)

2. Формирование соединений «один-ко-многим» и «многие-ко-многим», а также реализация запросов B1, B2 и B3 с использованием генераторов списков и функций высшего порядка изображены на рисунке 2.

```python
def main():
    one_to_many = [
        (b.title, b.price, s.name)
        for s in shops
        for b in books
        if b.shop_id == s.id
    ]

    many_to_many_temp = [
        (s.name, sb.shop_id, sb.book_id)
        for s in shops
        for sb in shops_books
        if s.id == sb.shop_id
    ]

    many_to_many = [
        (b.title, b.price, shop_name)
        for shop_name, shop_id, book_id in many_to_many_temp
        for b in books
        if b.id == book_id
    ]

    print('Задание В1')
    res_1 = [
        (title, shop_name)
        for title, _, shop_name in one_to_many
        if title.startswith('А')
    ]
    print(res_1)

    print('\nЗадание В2')
    res_2_unsorted = []
    for s in shops:
        s_books = list(filter(lambda i: i[2] == s.name, one_to_m
        if len(s_books) > 0:
            s_prices = [price for _, price, _ in s_books]
            s_price_min = min(s_prices)
            res_2_unsorted.append((s.name, s_price_min))

    res_2 = sorted(res_2_unsorted, key=itemgetter(1))
    print(res_2)

    print('\nЗадание В3')
    res_3 = sorted(many_to_many, key=itemgetter(0))
    print(res_3)
```

Рисунок 2 – Код функции main с реализацией запросов B1, B2 и B3 (rk1.py)

3. Результаты выполнения разработанной программы для запросов B1, B2 и B3 представлены на рисунке 3.

```
asffe@ASFProg:~/3sem/PPaLC/RK1$ python3 RK1.py
Задание B1
[('Алгоритмы: построение и анализ', 'Читай-город'), ('Архитектура компьютера', 'Библио-Глобус'), ('Анализ данных в Python', 'Мос
ковский Дом Книги')]

Задание B2
[('Московский Дом Книги', 1200), ('Библио-Глобус', 1800), ('Читай-город', 2400)]

Задание B3
[('Алгоритмы: построение и анализ', 2400, 'Читай-город'), ('Алгоритмы: построение и анализ', 2400, 'Республика'), ('Анализ данны
х в Python', 1450, 'Московский Дом Книги'), ('Анализ данных в Python', 1450, 'Республика'), ('Архитектура компьютера', 1950, 'Би
блио-Глобус'), ('Архитектура компьютера', 1950, 'Лабиринт'), ('Изучаем Python', 1800, 'Библио-Глобус'), ('Изучаем Python', 1800,
 'Буквоед'), ('Совершенный код', 2100, 'Московский Дом Книги'), ('Совершенный код', 2100, 'Буквоед'), ('Чистый код', 1200, 'Моск
овский Дом Книги'), ('Чистый код', 1200, 'Лабиринт')]
asffe@ASFProg:~/3sem/PPaLC/RK1$
```

Рисунок 3 – Результаты выполнения программы (RK1.py)

Вывод: в результате выполнения рубежного контроля были разработаны классы данных для предметной области «Книга – Книжный магазин» (вариант 16) и реализованы связи «один-ко-многим» и «многие-ко-многим». С применением функциональных возможностей языка Python (list comprehensions, функции filter и sortesd) были успешно выполнены запросы варианта В на фильтрацию, агрегацию и сортировку связанных данных.